

Mathematical Foundations of ETL, Spark & Streaming

Companion to Session 3 | Big Data Engineering Exercise Sheet (without solutions)

Parallelism, Windowing, Watermarks & Cost Models
for Batch and Streaming Pipelines

Document Contents

- Throughput–latency trade-off: batch vs. streaming.
- Spark parallelism: partitions, executors, and Amdahl’s Law.
- Windowing math: tumbling, sliding, and session windows.
- Watermarks and the probability of dropping late events.
- State size and memory budget for streaming aggregations.
- **Two exercises (statements only)** to be solved by the student.

Contents

1	Introduction	2
2	Latency vs. Throughput: Bounded vs. Unbounded Models	2
2.1	The two regimes	2
2.2	Why batch wins on throughput	2
2.3	Why streaming wins on latency	3
2.4	Numerical illustration	3
3	Spark Parallelism and Amdahl’s Law	3
3.1	The partition–task–executor mapping	3
3.2	Amdahl’s Law for Spark jobs	4
3.3	Choosing the partition count	4
4	Windowing in Structured Streaming	5
4.1	Three window types	5
4.2	How many windows does each event belong to?	5
4.3	State size for windowed aggregations	5
5	Watermarks: A Probabilistic View of Late Data	6
5.1	Modelling event lateness	6
5.2	Common lateness distributions	6
5.3	Choosing the watermark	6
6	A Cost Model for Batch vs. Streaming	6
6.1	Setting	6
6.2	Cost-equivalent latency	7
7	Exercises	8
8	Summary	10
8.1	Going further	10

1. Introduction

Session 3 connects three threads: the shape of a data pipeline (ETL vs. ELT, batch vs. streaming), the engine that runs it (Apache Spark with its driver/executor model), and the semantics of streaming (windows, watermarks, late events). Each of these is governed by a quantitative law – and engineers who can compute that law make better design choices than those who reason only by analogy.

We focus on five building blocks:

1. **Latency–throughput trade-off** → a batch job’s per-record overhead vanishes as the batch grows; a streaming job’s overhead is paid at every micro-batch.
2. **Spark parallelism** → task count, partition count, and Amdahl’s Law.
3. **Window arithmetic** → how often a record is counted under tumbling vs. sliding windows, and how state size scales.
4. **Watermark drop probability** → if event lateness has a known distribution, what is the probability that an event is dropped at threshold θ ?
5. **Cost model** → when does a streaming job become cheaper or more expensive than running batch every T minutes?

2. Latency vs. Throughput: Bounded vs. Unbounded Models

2.1 The two regimes

Recall the Session 3 distinction: **batch** processes bounded datasets; **streaming** processes an unbounded stream. Each regime has a different latency–throughput profile.

Definition – Per-Record Latency

For a job with start-up cost C (seconds) processing n records at a per-record cost c (seconds/record), the average per-record latency is

$$\ell(n) = c + \frac{C}{n}.$$

The fixed overhead C (cluster start-up, JVM warmup, query planning) is amortised across all n records. As $n \rightarrow \infty$, $\ell(n) \rightarrow c$.

2.2 Why batch wins on throughput

A nightly batch job processes n records in time $T_{\text{batch}} = C + nc$. Average throughput:

$$\lambda_{\text{batch}} = \frac{n}{C + nc} \xrightarrow{n \rightarrow \infty} \frac{1}{c}.$$

The fixed cost C is paid once. For large n , throughput approaches the theoretical maximum $1/c$.

2.3 Why streaming wins on latency

A streaming job processes records in micro-batches of size b every trigger interval Δt . The per-record latency is bounded by Δt :

$$\ell_{\text{stream}} \approx \Delta t + b c.$$

The throughput is $b/\Delta t$. Reducing Δt reduces latency but also reduces b (smaller windows accumulate fewer records), which raises the per-record overhead.

The Latency–Throughput Trade-off

For any pipeline:

$$\lambda_{\text{max}} = \frac{b}{\Delta t}, \quad \ell_{\text{min}} \approx \Delta t,$$

so reducing latency by a factor of k reduces maximum throughput by a factor of k for the same per-batch cost. Latency and throughput are coupled by the trigger interval.

2.4 Numerical illustration

Suppose $C = 30$ s (Spark application startup), $c = 10 \mu\text{s}/\text{record}$:

Regime	n or b	Avg. latency ℓ	Throughput
Batch (1B records)	10^9	$10 \mu\text{s} + 30 \text{ ns} \approx 10 \mu\text{s}$	$\sim 100,000 \text{ rec/s}$
Stream ($\Delta t = 5$ s, $b = 50,000$)	$5 \cdot 10^4$	~ 5 s	$10,000 \text{ rec/s}$
Stream ($\Delta t = 0.5$ s, $b = 5,000$)	$5 \cdot 10^3$	~ 0.5 s	$10,000 \text{ rec/s}$

Important Remark

This is why Lambda architectures historically separated batch and speed layers: the same compute infrastructure could not meet both sub-second latency and TB-per-hour throughput simultaneously. Modern engines (Spark Structured Streaming, Flink) have closed most of this gap, but the underlying trade-off remains.

3. Spark Parallelism and Amdahl's Law

3.1 The partition–task–executor mapping

A Spark job processes data through a chain of stages. Each stage operates on a number of partitions P . Spark schedules one task per partition, run by a slot on an executor (each executor has `spark.executor.cores` slots).

Definition – Effective Parallelism

With E executors and k cores per executor, total slots $S = E \cdot k$. For a stage of P partitions, the number of parallel tasks is

$$T_{\parallel} = \min(P, S).$$

The time to process the stage is

$$T_{\text{stage}} \approx \frac{P}{T_{\parallel}} \cdot t_{\text{task}} = \left\lceil \frac{P}{S} \right\rceil \cdot t_{\text{task}},$$

where t_{task} is the per-task execution time.

3.2 Amdahl's Law for Spark jobs

Real Spark jobs have a parallel section (the work spread across partitions) and a serial section (driver bookkeeping, query planning, final `collect`). If a fraction f of total work is parallelisable:

X¹ Theorem – Amdahl's Law

The maximum speed-up achievable using S parallel slots is

$$\text{Speed-up}(S) = \frac{1}{(1 - f) + \frac{f}{S}}.$$

As $S \rightarrow \infty$, the speed-up is bounded by $1/(1 - f)$.

A job with $f = 0.95$ can be sped up by at most a factor of $1/0.05 = 20$, no matter how many executors you add. This is why the driver is often the bottleneck once you have enough executors.

3.3 Choosing the partition count

A common rule of thumb: the number of partitions should be 2–4 times the total number of slots ($S = E \cdot k$).

- Too few partitions ($P < S$): some slots stay idle.
- Too many partitions ($P \gg S$): scheduling overhead dominates.
- $P \approx 2S$ to $4S$: load balancing absorbs stragglers without excessive task overhead.

Partition Sizing Guideline

$$P_{\text{recommended}} = 2S \text{ to } 4S \quad \text{with } S = E \cdot k.$$

4. Windowing in Structured Streaming

4.1 Three window types

Structured Streaming supports three window types. Let w denote the window length and s the slide interval (with $0 < s \leq w$).

- **Tumbling window:** $s = w$. Windows are non-overlapping; each event belongs to exactly one window.
- **Sliding window:** $s < w$. Windows overlap; each event belongs to multiple windows.
- **Session window:** dynamic length; closes after a gap of inactivity.

4.2 How many windows does each event belong to?

X¹ Theorem – Window Multiplicity

For a sliding window of length w and slide s with s dividing w , each event belongs to exactly

$$k = \frac{w}{s}$$

overlapping windows. For a tumbling window ($s = w$), $k = 1$.

Sketch. A sliding window starts at every multiple of s . An event at time t belongs to the window starting at time t_0 iff $t_0 \leq t < t_0 + w$. The number of valid starting times in any interval of length w is w/s . $\square$

Example. A 10-minute window sliding every 2 minutes assigns each event to $10/2 = 5$ windows. The aggregation work is $5\times$ that of a tumbling 10-minute window, because every record is counted five times.

4.3 State size for windowed aggregations

The streaming engine must keep state for every open window. Let θ be the watermark threshold (max event lateness tolerated). Then the engine keeps:

$$N_{\text{open windows}} \approx \left\lceil \frac{w + \theta}{s} \right\rceil.$$

With K distinct grouping keys, the state-memory footprint scales as

$$M_{\text{state}} \propto K \cdot \left\lceil \frac{w + \theta}{s} \right\rceil.$$

i Important Remark

A common production pitfall: setting $\theta = w$ (watermark equal to the window) doubles the open-window count and halves available memory. A watermark that is too generous quietly inflates state until the executor runs out of heap. Tighten θ to the actual observed event-time lag.

5. Watermarks: A Probabilistic View of Late Data

5.1 Modelling event lateness

Let D be the random variable representing how late a given event arrives behind the maximum event time observed so far. The watermark with threshold θ drops any event for which $D > \theta$.

Definition – Drop Probability

If D has cumulative distribution function F_D , the probability that a random event is dropped by a watermark of θ is

$$\mathbb{P}(\text{drop}) = \mathbb{P}(D > \theta) = 1 - F_D(\theta).$$

5.2 Common lateness distributions

Two empirically useful models:

- **Exponential lateness:** $D \sim \text{Exp}(\mu)$ with $\mathbb{E}[D] = 1/\mu$. Then $\mathbb{P}(D > \theta) = e^{-\mu\theta}$. Doubling θ squares the survival probability: $\mathbb{P}(D > 2\theta) = (e^{-\mu\theta})^2$.
- **Bounded lateness:** D uniform on $[0, D_{\max}]$. Then $\mathbb{P}(D > \theta) = \max(0, 1 - \theta/D_{\max})$. Setting $\theta = D_{\max}$ guarantees zero drops.

5.3 Choosing the watermark

Engineers usually pick θ such that the drop rate is below a service-level objective (e.g. $\mathbb{P}(\text{drop}) \leq 10^{-3}$).

Example. If event lateness is exponential with mean $\mathbb{E}[D] = 30$ seconds, what watermark threshold θ caps the drop rate at 0.1%?

$$e^{-\mu\theta} \leq 10^{-3} \iff \theta \geq \frac{\ln(10^3)}{\mu} = \mu^{-1} \cdot \ln(1000) = 30 \cdot 6.91 \approx 207 \text{ s.}$$

A watermark of ~ 3.5 minutes drops only 0.1% of events.

Watermark Sizing Formula

For exponential lateness with mean $1/\mu$ and target drop rate α :

$$\theta^* = \frac{1}{\mu} \cdot \ln\left(\frac{1}{\alpha}\right).$$

6. A Cost Model for Batch vs. Streaming

6.1 Setting

Suppose a company can run either:

- a batch job every T minutes that costs C_b per run, or
- a streaming job that runs continuously at cost rate r_s per minute.

6.2 Cost-equivalent latency

The hourly cost of the batch option is $\frac{60}{T} \cdot C_b$. The hourly cost of streaming is $60 \cdot r_s$.

X¹ Theorem – Streaming Becomes Cheaper When...

Streaming is more cost-effective than batch every T minutes whenever

$$60 \cdot r_s \leq \frac{60}{T} \cdot C_b \iff T \leq \frac{C_b}{r_s}.$$

Equivalently, batch is preferred when the desired refresh interval T exceeds C_b/r_s minutes.

Example. If a batch run costs $C_b = \$5$ and streaming costs $r_s = \$0.10/\text{minute}$, the cost crossover is at $T = 5/0.10 = 50$ min. For latency tighter than 50 min, streaming wins; for refresh intervals longer than 50 min, batch wins.

7. Exercises

The two exercises that follow combine the building blocks above. Every numerical value needed is given in the statement.

Exercise 1 – Spark Parallelism, Partition Sizing, and Amdahl’s Law

A Spark batch job processes a $T = 500$ GB dataset stored in $P_0 = 4,000$ HDFS blocks of 128 MB each. The job runs on a cluster of $E = 20$ executors, each with $k = 4$ cores and $m = 16$ GB of memory.

Part A. Parallelism and partitioning.

- Compute the total number of parallel slots S .
- Spark loads one partition per HDFS block by default. How many waves of parallel tasks are required to process all $P_0 = 4,000$ partitions?
- Suppose each task takes $t_{\text{task}} = 6$ s. Compute the total stage time T_{stage} .
- The team is considering coalescing the input down to $P_1 = 80$ partitions before processing. How many waves does this require, and what is the new T_{stage} assuming task time scales linearly with partition size (i.e. each new task takes $t'_{\text{task}} = 6 \cdot (P_0/P_1)$)?
- Comment on whether the coalesce is helpful. Compute the per-task data size for both partition counts and identify any concern.

Part B. Amdahl’s Law. The job has a serial driver section (DAG planning, final `collect()`) representing a fraction $1 - f$ of total work; the rest is fully parallelisable.

- For $f = 0.90$ and $S = 80$ slots, compute the speed-up vs. a single slot.
- What is the theoretical maximum speed-up as $S \rightarrow \infty$?
- The team doubles the cluster to $S' = 160$ slots. What is the new speed-up? What relative improvement does this yield compared to $S = 80$?
- For what value of f would a single doubling from $S = 80$ to $S = 160$ yield exactly a $1.5\times$ improvement in speed-up?

 Exercise 2 – Windowing, Watermarks, and Cost Trade-offs

A streaming pipeline ingests IoT sensor readings from $K = 50,000$ distinct devices into Spark Structured Streaming. Each reading is a 120-byte row.

Part A. Window arithmetic. The team computes a rolling average sensor value per device using a sliding window of length $w = 10$ minutes and slide $s = 2$ minutes, with watermark threshold $\theta = 4$ minutes.

- (a) How many overlapping windows does each event belong to?
- (b) How many open windows does Spark maintain at any time?
- (c) Each window keeps one 24-byte aggregate per device (running sum + count). Estimate the total state memory required, in MB.
- (d) The team considers switching to a tumbling 10-minute window. What is the new state memory? By what factor is it smaller?

Part B. Watermarks and late events. The team measures that the event lateness D is approximately exponential with mean $\mathbb{E}[D] = 45$ seconds.

- (e) Compute the rate parameter μ of the exponential distribution.
- (f) With the current $\theta = 4$ min watermark, compute the probability that a given event is dropped.
- (g) The product team requires a drop rate of at most $\alpha = 10^{-4}$ (one in ten thousand). What is the smallest watermark threshold θ^* that meets this target? Express your answer in minutes.
- (h) How much does the new θ^* change the state memory required, compared to $\theta = 4$ min, for the original sliding window? (Assume sliding window with $w = 10$, $s = 2$.)

Part C. Batch vs. streaming cost model. A batch alternative would re-run the same aggregation every T minutes at cost $C_b = \$8$ per run. The streaming pipeline costs $r_s = \$0.20$ per minute to keep alive.

- (i) At what refresh interval T^* does the streaming option become cheaper than batch?
- (j) The product team requires the result to be no more than 5 minutes stale. Which option is cheaper, and by how much per hour?

8. Summary

Key Formulas to Remember

Concept	Formula
Per-record latency	$\ell(n) = c + C/n$
Latency–throughput coupling	$\lambda_{\max} = b/\Delta t, \quad \ell_{\min} \approx \Delta t$
Spark slots	$S = E \cdot k, \quad T_{\parallel} = \min(P, S)$
Stage time	$T_{\text{stage}} = \lceil P/S \rceil \cdot t_{\text{task}}$
Recommended partitions	$P \in [2S, 4S]$
Amdahl's Law	$\text{Speed-up}(S) = 1/((1 - f) + f/S)$
Window multiplicity	$k = w/s$ (sliding); $k = 1$ (tumbling)
Open windows	$N_{\text{open}} = \lceil (w + \theta)/s \rceil$
Drop probability (exp. lateness)	$\mathbb{P}(\text{drop}) = e^{-\mu\theta}$
Watermark sizing	$\theta^* = (1/\mu) \cdot \ln(1/\alpha)$
Streaming-vs-batch crossover	$T^* = C_b/r_s$

8.1 Going further

- Akidau, T., Chernyak, S., Lax, R. *Streaming Systems*. O'Reilly, 2018. The reference text on windowing and watermarks.
- Chambers, B., Zaharia, M. *Spark: The Definitive Guide*. O'Reilly, 2018. Chapters on the Catalyst optimiser and Structured Streaming.
- Amdahl, G. *Validity of the single processor approach to achieving large scale computing capabilities*, AFIPS, 1967.
- Kleppmann, M. *Designing Data-Intensive Applications*, Ch. 10–11 (Batch and Stream Processing).